

Fig. 9: Taxonomy of Efficient PEFT Design

heads, each affecting the head attention output in SA; and *Scaled Parallel Adapter*, which adds a scaling term after the parallel adapter layer, similar to LoRA. Extensive experimentation revealed that the most effective configuration involves using prefix-tuning in the SA layer and the scaled parallel adapter in the FFN layer, which is called MAM Adapter. **LLM-Adapters** [78] builds an easy-to-use framework that incorporates various PEFT techniques into LLMs. Through comprehensive benchmarking across multiple datasets, the study reveals several key insights: (1) The most effective locations for series adapters, parallel adapters, and LoRA are after the MLP layers, alongside the MLP layers, and simultaneously following the Attention layers and MLP layers, respectively. (2) Smaller LLMs utilizing PEFT can achieve competitive or even superior results on certain tasks when compared to their larger counterparts. (3) With appropriate in-distribution fine-tuning data, smaller models are capable of surpassing larger models in task-specific performance.

Several studies leverage neural architecture search (NAS) to find better PEFT combination approaches. For example, **NOAH** [222] discovers that different PEFT configurations are specifically tailored for different tasks. To address this issue, NOAH employs NAS to identify the most effective PEFT configurations for each dataset. Specifically, NOAH’s searching space encompasses three PEFT methods: Adapter, LoRA, and Visual Prompt Tuning (VPT). It utilizes AutoFormer [20], a one-shot NAS algorithm, for the efficient discovery of optimal prompt modules. In a related vein, **AUTOPEFT** [231] first establishes a searching space that includes serial adapters, parallel adapters, and prefix tuning. After that, they propose an effective NAS methods based on a high-dimensional multi-dimensional Bayesian optimisation [47]. Both NOAH and AUTOPEFT demonstrate the capability of NAS in enhancing PEFT configurations across a variety of tasks.

IV. EFFICIENT PEFT DESIGN

Processing latency and peak memory overhead are pivotal factors to consider from a computational standpoint. This section introduces a key characteristic in LLMs aimed at balancing between latency and memory usage (Section IV-A). Following this, we explore strategies for developing efficient PEFT methods to address computational challenges, including **PEFT pruning** (Section IV-B), **PEFT quantization** (Section IV-C), and **memory-efficient PEFT techniques** (Section IV-D), each designed to enhance model performance while minimizing resource consumption. It is noteworthy that quantization inherently addresses memory overhead concerns.

However, given its distinct characteristics, we address these quantization methods separately rather than incorporating them under the memory-efficient PEFT section.

A. KV-cache Management for PEFT Efficiency

The core of the LLMs model lies an autoregressive Transformer model, depicted in Figure 2. When we look at autoregression characteristic, it becomes a major challenge in designing an inference system, because every time a new token is generated, the entire LLM model has to transfer all the weights from different memories to the memory of the graphics processor, which is very unfriendly to single-user task scheduling or multi-user work-load balance. The challenging part of serving the auto-regressive paradigm is all previous sequences have to be cached and saved for the next proceeding iteration, the cached activation generated from the previous sequences is stored as the Key-Value Cache (KV-cache).

The storage of KV-cache will cost both memory space and IO performance, yielding in workload memory-bounded and under-utilizing the computation power of the system. Previous works proposed a series of solutions like KV-cache control management [98] or KV-cache compression [159] to improve throughput or reduce latency. When designing PEFT methods, it is crucial to consider the characteristics of the KV-cache to complement its features. For instance, when applying soft prompts in the inference phase, efficiently leveraging the KV-cache for these additional inputs can help accelerate response times by ensuring prompt-related data is readily accessible.

B. Pruning Strategies for PEFT

The inclusion of pruning can substantially enhance the efficiency of PEFT methods. In particular, **AdapterDrop** [154] explores the removal of adapters from lower transformer layers and multi-task adapters in AdapterFusion [145], which shows that the pruning can improve the training and inference efficiency with minimal decrease in performance. **SparseAdapter** [69] investigates different pruning methods and finds that high sparsity ratios (80%) can outperform standard adapters. Additionally, the *Large-Sparse* configuration, which increases the bottleneck dimension while maintaining a constant parameter budget (e.g., doubling dimensions with a 50% sparsity), substantially enhances the model’s capacity, resulting in improved performance. **SPLoRA** [72] adopts channel-based pruning to the LoRA weights W_{down} and W_{up} . This pruning affects not only the source weights W_0 , but also the LoRA parameters W_{up} and W_{down} . Similarly,

LoRAPruning [216] adopts structured pruning not only to the pretrained model weights but also to the LoRA weights. In contrast to unstructured LoRA pruning methods, which primarily focus on sparsifying model weights while leaving LoRA weights dense, thus making weight merging challenging to achieve, LoRAPruning enables the weights to be merged easily. Additionally, this work also introduces a novel criterion that utilizes LoRA’s gradients as an approximation of the gradients for the pre-trained weights, enabling the estimation of weight importance. **ProPETL** [210] constructs a single shared *prototype* (e.g., adapter, prefix, or LoRA) across layers and tasks. In addition, ProPETL learns binary masks to prune different sub-networks in different layers and tasks. As a result, the parameters can be reused in across layers and tasks, largely increasing the parameter efficiency.

C. Quantization Strategies for PEFT

Quantization serves as another popular technique for improving computational efficiency and reduce memory usage. For example, by investigating the loss landscape of adapters, **BI-Adapter** [87] finds that adapters are resistant to noise in parameter space. Building on this insight, the authors introduce a clustering-based quantization approach. Remarkably, they demonstrate that a 1-bit quantization of adapters not only minimizes storage requirements but also achieve superior performance among all precision settings. **PEQA** (Parameter-Efficient and Quantization-aware Adaptation) [94] uses a two-stage pipeline to achieve parameter-efficient and quantization-aware fine-tuning. In the first stage, the pre-trained FFN weight matrix $W \in \mathbb{R}^{n \times m}$ is quantized to $W = s \cdot \bar{W}$, where $s \in \mathbb{R}^{n \times 1}$ represents per-channel scales and $\bar{W}$ denotes the quantized weight. In the second stage, $\bar{W}$ remains fixed, and fine-tuning is only conducted on s . This approach not only ensures memory efficiency but also facilitates parameter efficiency. **QLoRA** [37] proposes several novel techniques, including a 4-bit NormalFloat, a Double Quantization, and a Paged Optimizers, to backpropagate a 4-bit quantized pre-trained language model into LoRA. These techniques enable the fine-tuning for a 65B language model on a single 48GB GPU while maintaining similar performance to the full 16-bit fine-tuning. Similar to the original implementation [76], QLoRA attaches the fixed zero initialized LoRA weights to the quantized pre-trained model as the training start point. However, when applying the extreme low-bit (e.g., 2-bit) quantization, the huge quantization error can adversely impact the initialization of LoRA fine-tuning, i.e., $\text{quantization}(W_0) + W_{\text{down}}W_{\text{up}} \neq W_0$ where $W_{\text{down}} = \mathbf{0}$, which will harm the fine-tuning performance as shown in [112]. To solve this, several quantization strategies are proposed to eliminate the quantization error. For example, **LoftQ** (LoRA-Fine-Tuning-aware Quantization) [108] presents an innovative framework that provides a superior initialization point of quantized backbone weights and LoRA weights for subsequent LoRA fine-tuning. This approach addresses the discrepancies caused by quantization through the optimization of a Frobenius norm objective during network initialization, which takes both the LoRA weights and the quantized pre-trained backbone into

consideration. LoftQ exhibits superior performance in 2-bit quantization over QLoRA, as well as greater generalization for downstream tasks. **LQ-LoRA** [60] uses an iterative algorithm inspired by robust principal components analysis [234], [183] which decomposes the weight W_0 such that $W_0 \approx Q + L_1L_2$ to resolve the inaccuracy caused by the quantization error, where Q is the quantized component which remains fixed and L_1L_2 is the trainable low-rank component. Moreover, this approach leverages integer linear programming to determine a mixed quantization strategy, enabling dynamic quantization configurations for each weight matrix while adhering to a predetermined total bit rate limit. **QA-LoRA** [195] address another limitation of QLoRA, which struggles to preserve its quantized property post fine-tuning. In QLoRA, the quantized pre-trained weight (NF4) have to be recovered to FP16 to match the LoRA weight precision (FP16) during weight merging. Instead, QA-LoRA uses INT4 quantization and introduces group-wise operators to enable quantization during inference stage, therefore improve the efficiency and accuracy compared with QLoRA. **BitDelta** [118] introduces a novel 1-bit post-training quantization method that acts on the weight delta between a fine-tuned model and its underlying pre-trained model. Specifically, given the weight matrices W_{fine} and W_{base} from the fine-tuned and base models respectively, the weight delta $\Delta = W_{\text{fine}} - W_{\text{base}}$ is binarized as $\hat{\Delta} = \alpha \odot \text{Sign}(\Delta)$. Here, α , a high-precision scalar, is initialized based on the mean absolute delta value $\alpha = \frac{1}{nm} \sum_{ij} |W_{ij}|$, with $\text{Sign}(\cdot)$ indicating the sign of Δ . BitDelta further calibrates the scaling factors via distillation on a compact calibration dataset, while the binary matrices remain unchanged. This approach notably streamlines the deployment of multiple fine-tuned models on shared servers by utilizing a singular full-precision base model alongside efficiently batched 1-bit deltas.

D. Memory-efficient PEFT Methods

Fine-tuning the full LLMs necessitates substantial training memory owing to their considerable size. While most PEFT methods primarily target parameter efficiency, they still incur a significant memory overhead during training because gradient computation and backpropagation are still necessary for these methods. For example, prevalent PEFT techniques such as adapters and LoRA can only reduce memory usage to approximately 70% compared to full model fine-tuning according to some literatures [167], [88]. From a computational perspective, memory efficiency also remains a critical factor that cannot be overlooked.

To improve memory efficiency, various techniques have been developed to minimize the need for caching gradients for the entire LLM during fine-tuning, thereby reducing memory usage. For example, both **Side-Tuning** [213] and **LST** (Ladder-Side Tuning) [167] introduces a learnable network branch parallel to the backbone model. By channeling the backpropagation exclusively through this parallel branch, it circumvents the need to store gradient information for the main model’s weights, thus markedly reducing memory requirements during training. Similarly, **Res-Tuning** [85] disentangles the PEFT tuners (e.g., prompt tuning, adapter) from the

backbone model. On top of the disentanglement, a memory-efficient fine-tuning framework named Res-Tuning-Bypass is proposed, which generates a bypass network in parallel with the backbone model by removing the data flow from the decoupled tuners to the backbone. This eliminates the requirement for gradient caching within the backbone model during backpropagation. **MEFT** [112] (memory-efficient fine-tuning) is an approach inspired by the *reversible model* [55]. During the training of a reversible model, intermediate activations are not required to be cached in the forward pass. During backpropagation, they can be recalculated from the final output. To save the memory during fine-tuning, MEFT investigates how to transform an LLM to its reversible counterparts without additional pre-training. A critical aspect of this transformation is the careful initialization of newly-introduced parameters in the pre-trained models. MEFT demonstrates the importance of the parameter initialization, and suggests that these parameters must be initialized in a manner that preserves the pre-trained model’s starting point, ensuring that the fine-tuning of the modified model achieves performance on par with full fine-tuning methods. With this key consideration, MEFT introduces three distinct methods, each significantly curtailing the memory demands traditionally required for storing activations. **LoRA-FA** [215] addresses a limitation about memory overhead in LoRA fine-tuning. During training, LoRA modules still require high activation memory consumption. This is because, during backpropagation, large input activations must be stored during the forward pass to compute gradients. LoRA-FA resolves this issue by freezing both the pre-trained weights W_0 and the projection-down weights W_{down} , and only updating the projection-up weights W_{up} . Consequently, the input activation h_{in} no longer needs to be stored, as the intermediate activation $W_{\text{down}}h_{\text{in}}$ is adequate for gradient computation for W_{up} . Given that $r \ll d$, the memory requirement for activations in LoRA-FA can be significantly reduced.

To further reduce memory usage during fine-tuning, some methods attempt to circumvent backpropagation within LLMs to address this issue. **HyperTuning** [146] employs a HyperModel to generate PEFT parameters using only fewshot examples. This approach demonstrates results comparable to those obtained through full model fine-tuning. **PEFT Plug-in** [88] first trains PEFT modules on small language models, which is more memory efficient compared to training on large ones. Subsequently, the research introduces a suite of techniques for seamlessly integrating these trained PEFT modules into LLMs during inference. This strategy effectively circumvents the necessity of gradient-based optimization directly on the larger models, resulting in substantial memory savings. However, it is important to note that both HyperModel and PEFT Plug-in still require additional model training, and this training cost cannot be entirely overlooked. **MeZO** [131] introduces a memory-efficient zeroth-order (ZO) optimizer for LLMs. Unlike conventional PEFT techniques, which rely on backpropagation to compute gradients for updating model parameters, MeZO fine-tunes LLMs through only forward passes. It accomplishes this by employing a ZO gradient estimator to calculate the gradient. Notably, MeZO implements an in-place solution for the classic ZO gradient estimator, effectively mitigating memory con-

sumption during inference execution. This innovative approach allows for efficient fine-tuning of LLMs containing 30 billion parameters on a single GPU with 80GB of memory, all while maintaining performance that is comparable to fine-tuning using backpropagation. Furthermore, it can substantially decrease storage demands in comparison to the traditional PEFT methods such as LoRA and Adapter.

V. PEFT FOR DNNs OF OTHER APPLICATIONS

In Section III, we outlined four categories of PEFT methods along with their improvements. Nonetheless, our discussion did not fully extend to the utilization or adaptation of PEFT techniques beyond traditional architectures (e.g., LLMs) or standard benchmarks (e.g., the GLUE dataset), where the majority of the discussed PEFT methods are applied. Therefore, in this section, we will highlight and discuss several most representative works that leverages PEFT strategies for various downstream tasks. We do not aim to cover all PEFT application scenarios in this section. Our objective is to showcase the significant influence of PEFT within various research domains, and demonstrate how to optimize and tailor general-purpose PEFT methods to achieve enhanced performance in specific models or tasks.

Typically, fine-tuning happens when adapting a pre-trained backbone model to specialized downstream tasks. To this end, this section organizes the discussion around various model architectures, which include: LLM, Vision Transformer (ViT), Vision-Language Alignment Model (VLA), and Diffusion model. Within each architectural category, the discussion is further classify based on different downstream tasks.

A. PEFT for LLMs – Beyond the Basics

Instead of common tasks in NLP such as NLU and NLG, PEFT techniques boast a wide array of applications across diverse scenarios. PEFT has been successfully implemented in commonsense question answering [82], [229], multi-level implicit discourse relation recognition [227], out-of-distribution detection [141], privacy protection [142], [190], federated learning [15], and social biases mitigation [106]. In this section, we pay more focus on three representative downstream tasks: visual instruction following, continual learning, context window extension.

1) *Visual Instruct Following*: Several studies, including VL-BART [26], MiniGPT-4 [237], and LLaVA [115], have successfully extended the capabilities of LLMs, initially designed for pure text, to comprehend and generate responses to visual inputs. These enhanced models, namely visual instruct-following LLMs, can process both images and text to produce textual responses, which can be benchmarked on tasks such as image captioning [152], [204], [172], [74] and visual question answering (VQA) [176], [188], [4]. However, these methods fine-tune the entire LLM to learn the visual representations, which can be inefficient in both time and memory. Therefore, it’s natural to apply PEFT techniques in the fine-tuning of visual instruct-following LLMs. An earlier work **VL-Adapter** [168] directly applies several PEFT methods (Adapter [75], Hyperformer [129] and Compacter [91]) on

VL-BART [26] then benchmarks them on several image-text and video-text tasks. Results show that vanilla adapters are the best among them, which can achieve performance on par with full fine-tuning. However, considering the functionality gap between the encoders and decoders in VL-BART, directly assign identical modular modifications will lead to suboptimal performance. Therefore, **VL-PET** [79] selectively integrates PEFT modules into different components of the encoder and decoder. They also introduces a granularity-controlled mechanism for finer-grained control.

To adapt the recently prevalent LLaMA model, **LLaMA-Adapter** [220] prepends a set of learnable prompts (similar to prefix tuning) to the input tokens in LLaMA’s higher transformer layers. To avoid the unstable fine-tuning with large loss values at early training stages, instead of the randomly initialized weights of other PEFT methods, LLaMA-Adapter adopts a zero-initialized attention mechanism, which learns a zero-initialized gating factor to adaptively control the contribution of adaptation prompts to the word tokens. This can maintain the fine-tuning starting point the same as the original model and progressively inject new knowledge into the model, where similar idea can be found in MEFT [112] and LoftQ [108] discussed earlier. To represent visual information, LLaMA-Adapter extract multi-scale global image features using CLIP image encoder than projects them to linguistic embedding space. After that, the feature is element-wisely added onto the adaptation prompts at all inserted transformer layers. LLaMA-Adapter only introduces 1.2M learnable parameters in LLaMA-7B, and costs less than one hour for fine-tuning on 8 A100 GPUs. A following work **LLaMA-Adapter V2** [52] demonstrates that the simple multimodal fusion in LLaMA-Adapter cannot generalize to more challenging open-ended multimodal reasoning tasks, where the visual cues tend to dominate the adaptation prompts than the language instruction data. To address this, LLaMA-Adapter V2 decouples the learning of instruction-following ability (to generate long language responses) and vision-language alignment to avoid interference between visual and language fine-tuning. Specifically, LLaMA-Adapter V2 sets disjoint parameter groups which are respectively learned from image-text pairs and language instruction data. The visual adaptation prompts are inserted in the early stage of LLM, while the language adaptation prompts keeps at the higher transformer layers similar to LLaMA-Adapter. Additionally, LLaMA-Adapter V2 introduces more learnable parameters and several expert systems (e.g., captioning, detection, and OCR) to enhance multimodal performance. **LayerNorm Tuning** [225] adjust only the weights of the LayerNorm within each attention block. This straightforward technique can achieve comparable or even better performance than the finetuning, while offer about 10× more parameter efficiency than LoRA.

2) *Continual Learning (CL)*: CL aims to learn a sequence of new tasks over time within one single model, which has broad application in scenarios such as dialogue systems [100], information extraction systems [14], and question answering systems [201]. The main challenge in CL is catastrophic forgetting [95]. A popular practice, called architecture-based methods, tackles the CL by maintaining task-specific param-

eters in the model for each new task. Therefore, it’s natural to leverage PEFT methods for CL tasks [128], [238], [33], [110]. For example, **AdapterCL** [128] parameterizes each new task using residual adapters. During testing, since the task-id is not provided, AdapterCL uses an entropy-based classifier to select which adapter to use for accomplishing specific task. **CPT** (Continual Prompt Tuning) [238] trains a soft prompt for each task. Instead of training soft prompts from scratch, CPT proposes a series techniques (continual prompt initialization, query fusion, memory replay, and a memory-guided technique) to achieve knowledge transfer from preceding and subsequent tasks. **O-LoRA** (orthogonal low-rank adaptation) [178] employs a strategy of learning distinct tasks within separate low-rank vector subspaces that are kept orthogonal to each other in order to minimize interference. This approach can effectively reducing catastrophic forgetting during the acquisition of new tasks.

3) *Context Window Extension*: LLMs are typically trained with a pre-defined context size. For example, LLaMA and LLaMA2 have pre-defined context sizes of 2048 and 4096 tokens, respectively. The positional encoding RoPE has weak extrapolation properties [22], which means the performance drops obviously given an input length exceeds the pre-defined context length. To solve this, a naive solution is to fine-tune a pre-trained LLM to longer context. However, this escalates computational costs quadratically with context size, straining memory and processing resources. To address this, **LongLoRA** [24] proposes to fine-tune a pre-trained LLM using LoRA to enlarge the context size. To reduce the perplexity gap between LoRA tuning and full fine-tuning, LongLoRA also opens embedding and normalization layers for training. In order to further improve training efficiency in long context scenario, LongLoRA further introduces a novel shifted sparse attention (S^2 -Attn) as an efficient substitute for standard self-attention during training. A subsequent study **LongQLoRA** [198] combines the advantages of LongLoRA with QLoRA and Position Interpolation [165] to save GPU memory. This work successfully extends context length of LLaMA2-13B from 4096 to 8192 on a single V100 with 32GB memory.

In addition to limited training-stage sequence length, real-world system memory constraints introduce another critical bottleneck to the context window. Specifically, the capacity of the KV-cache is curtailed by available system memory. For example, a 30B parameter LLM operating with an input length of 1024 and a batch size of 128 might necessitate up to 180GB for the KV-cache [223], thereby restricting the feasible size of the context window. In response to this, some strategies have resorted to quantizing the KV cache [159], [36], but quantization will certainly compromises performance. To effectively counteract this issue without significant loss, **GEAR** [90] presents a novel approach by employing a low-rank matrix to capture the majority of coherent bases of quantization error, complemented by a sparse matrix that addresses errors from outlier entries, thus efficiently minimizing approximation errors.

B. PEFT for ViTs

ViT [40] has emerged as a powerful backbone model in the recent computer vision community. In ViT model, images are treated as sequences of fixed-size patches analogous to how LLM uses discrete tokens. These patches undergo linear embedding and then receive positional encodings. Subsequently, they are processed through standard Transformer encoders. The training of ViT can be supervised [40], [164] or self-supervised [23], [68], and ViT can achieve superior performance when training with more data and using larger model size [35]. However, such scaling up inevitably escalates training and storage costs. Therefore, similar to LLMs, PEFT widely implemented in various downstream tasks, such as dense prediction [25], continual learning [182], [53], deep metric learning [151]. Here, we focus on two typical tasks to showcase the involvement of PEFT: image classification and video recognition.

1) *Image Classification*: Image classification on targeted visual datasets is a very common demand and has extensive applications, while pre-train then fine-tuning paradigm serves as a widespread strategy. A variety of methods leverage PEFT techniques to achieve efficient model tuning [84], [25], [21], [86]. For instance, **AdaptFormer** [21] inserts adapter modules in parallel to the FFN of the original ViT model for visual recognition tasks. **VPT** (Visual Prompt Tuning) [84] prepends a small amount of task-specific parameters into the input sequence of each Transformer layer. When applying ViT to downstream tasks, only these added parameters and the classification head are set to trainable. **The work by** [203] notices that compared with supervised ViT, VPT often underperforms with self-supervised ViT. Further analysis demonstrates that different pre-trained methods and downstream tasks have varying degrees of dependency on transformer blocks at different locations. To tackle this issue, the research introduces adaptable gates for ViT blocks. These gates dynamically modulate the contribution of prompt tokens to ViT blocks, allowing for a more targeted adaptation of the model to the task at hand.

2) *Video Recognition*: Several works consider the more challenging adaptation problem that transfer ViT to downstream tasks that has a much larger domain gap. For example, **ST-Adapter** (Spatio-Temporal Adapter) [143] and **AIM** [200] both insert adapters layers into pre-trained ViT blocks. Their primary goal is to model spatial-temporal information, thereby enabling efficient adaptation of ViTs from image models to video tasks. Notably, both methodologies have exhibited performance that surpasses traditional full-model fine-tuning approaches.

C. PEFT for VLAs

Vision-Language alignment models (VLA), such as CLIP [149], ALIGN [83], DeCLIP [107], and FLAVA [162], are designed to learn a good image and text features which can be aligned within a unified representation space. Each VLA typically consists of separate image and text encoders that extract respective features. Contrastive learning is leveraged in these models to effectively align the image and text features. Fine-tuning is leveraged to improve the performance of VLA

in specific dataset or tasks, but fine-tuning the full model is computationally intensive. For instance, fine-tuning CLIP RN50x64 requires a batch size of 32,768 and 18 days of training on 592 V100 GPUs [149]. Moreover, full fine-tuning on smaller datasets often leads to catastrophic forgetting [95]. In response to these challenges, and drawing inspiration from the success of PEFT techniques in NLP, a range of PEFT strategies have been proposed and implemented in VLA models, such as semantic segmentation [193], [205], [196], point cloud understanding [219], [241], [181], [81], video understanding [89], [138], [114], visual reasoning [63], [41], temporal action detection [137], to name a few. This section will focus on one common task that uses VLAs: open-vocabulary image classification.

1) *Open-vocabulary Image Classification*: In open-vocabulary image classification, earlier works design class-specific prompts, e.g., *a photo of a [CLASS]*, for each category, and ranks images based on their similarity to these textual descriptions. **CoOp** (Context Optimization) [233] replaces the handcrafted text prompt with learnable vectors, while keep the entire VLA fixes during training. **CoCoOp** (Conditional Context Optimization) [232] builds on this by tackling CoOp's limitations in generalizing to unseen classes. It introduces a lightweight neural network that generates an input-specific context token, dynamically adapting the prompt based on each image, thereby enhancing generalizability, but at the cost of increased computational demands due to the instance-aware operation. **ProGrad** [236] addresses the over-fitting risk in CoOp in few-shot setting by regularizing the soft prompt updates whose gradient is aligned to the general knowledge only updates the prompt whose gradient is aligned (or non-conflicting) to the general knowledge offered by the original prompt. **MaPLe** [93] notes that existing methods learn prompts either in the language or in the vision branch of CLIP, which is not efficient to leverage the multimodal nature of VLAs. To address this, MaPLe proposes branch-aware hierarchical prompts that simultaneously adapt both language and vision branches, and achieves superior performance. **TPT** (test-time prompt tuning) [161] studies prompt tuning on the fly without additional training samples. Specifically, during inference, TPT first augments the input image into various views, which are then utilized to tune the learnable prompts. The primary training objective is to ensure the VLA can generate consistent responses when faced with these differing views. A following work **DiffTPT** [44] further enhances the data diversity of test samples through diffusion models.

In another direction, several studies explore the usage of adapters in VLA. For example, **CLIP-Adapter** [51] integrates residual-style adapters after CLIP's text and visual encoders. Therefore, unlike CoOp and CoCoOp, CLIP-Adapter avoids the gradients backpropagation through CLIP's encoders, leading to reduced computational requirements in terms of both training memory and time. **Tip-Adapter** [218] adopts the same design with CLIP-Adapter. Different from CLIP-Adapter, the weights of adapter is obtained in a training-free manner from a query-key cache model [140], [57] constructed from fewshot supervisions in a non-parametric manner. As a result, Tip-

Adapter exhibits great efficiency compared to CLIP-Adapter’s SGD training process.

D. PEFT for Diffusion Models

Diffusion models [73], [163] are a class of generative models that learn to generate data by transforming random noise into a structured output by a progressive denoising process. During training, diffusion models learn to reverse the noise added to training data using a denoising network, while in inference, they start from noise, using denoising network to iteratively create data that mirrors the same distribution as the training examples. Diffusion models has various applications [62], [199], [32], [38], [155], while the most notable is stable diffusion [153], which bridges the gap between text and image with its robust capability to generate coherent and contextually relevant images directly from textual descriptions. Numerous studies leverage PEFT techniques to adapt a pre-trained diffusion model for downstream tasks, including accelerating sampling speed [125], [11], text-to-video adaptation [186], [191], text-to-3D adaptation [209], etc. This section mainly focus on two scenarios: integrating additional input modalities beyond mere text-based conditioning, and customizing content generation based on pre-trained diffusion model.

1) *Additional Input Control*: To incorporate additional input modalities (e.g., layout, keypoints) while retaining the extensive knowledge in the pre-trained model, **GLIGEN** introduces a novel approach, which maintains the original model’s weights intact and integrates new, trainable gated Transformer layers [2] that take in the new grounding input. The resulting model can not only accurately represent the grounding conditions but also produce high-quality images. Remarkably, the model can also generalize well to unseen objects during inference. **ControlNet** [214] fine-tunes a trainable copy of the encoding layers from Stable Diffusion while locks its pre-trained parameter weights. The fixed original model and the trainable copy are bridged through zero convolution layers. These layers, starting with zero-initialized weights, are designed to progressively adapt during training, ensuring that harmful noise does not affect the pre-trained features of Stable Diffusion at the beginning of training. This refined model is capable of conditioning on a variety of inputs such as Canny edges, Hough lines, user scribbles, human key points, segmentation maps, shape normals, depths, etc. **Concept Sliders** [50] introduces a plug-and-play LoRA adaptors to allow precise editing of concepts (e.g., age, smiling) within a diffusion model. **T2I-Adapter** [136] introduces a lightweight adapter model designed to align external control signals with the internal knowledge of text-to-image diffusion models. This adapter enables precise manipulation through structural control (e.g., sketch, depth map, semantic segmentation map, and keypose), color control (e.g., hue and color distribution), and integrating various controls by composing multiple adapters.

2) *Customized Generation*: The effectiveness of text-to-image diffusion models is limited by the user’s ability to articulate the desired target through text descriptions. For instance, it is difficult to describe the precise features of an

innovative toy car which is not encountered during large-scale model training. Consequently, the objective of customized generation is to enable the model to grasp new concepts from a minimal set of user-supplied images. **Textual Inversion** [49] addresses this by finding a new pseudo-word S_* (similar to soft prompt discussed in Section III-A2) that represent new, specific concepts in the textual embedding space of pre-trained text-to-image diffusion models. The pseudo-word S_* is optimized via the original optimization goal in diffusion models given a small image set (typically 3-5 images) depicting the concept, and the pre-trained model is leaved untouched. During inference, S_* can be treated like any other word and compose with other textual queries (e.g., “a photo of S_* on the beach”). **Custom Diffusion** [97] tackles a more challenging setting: compositional fine-tuning of multiple concepts. It fine-tunes only the W_k, W_v mapping from text to latent features in attention layers, which yields superior performance in multi-concept learning scenarios. Additionally, during fine-tuning, Custom Diffusion prevents model forgetting by introducing a small set of real images with captions akin to the target, alongside employing augmentation for faster convergence and improved results. **IP-Adapter** [202] identifies limitations in current approaches (e.g., ControlNet and T2I-Adapter) which project condition signals into the cross-attention modules. When handling image conditions aiming at controlling content, these methods unable to generate images faithful to the prompted image. The issue stems from that merging image features and text features within cross-attention layers loses image-specific information, leading to only coarse-grained controllable generation such as image style rather than image content. To overcome this, IP-Adapter introduces a novel decoupled cross-attention mechanism to distinguish between text and image features. IP-Adapter adds an additional cross-attention layer exclusively for image features in each cross-attention layer, and only the parameters of the new cross-attention layers are trained.

VI. SYSTEM DESIGN CHALLENGE FOR PEFT

A. System design for PEFT

In this section, we begin by providing a concise overview of cloud-based PEFT systems. Following this, we present the corresponding metrics employed for evaluating the system performance. Additionally, we present three prospective utilization scenarios to illustrate the challenges in system design.

1) *Centralized PEFT Query Serving*: Cloud providers have recently introduced a range of LLM services aimed at providing user applications through application programming interfaces (APIs) [139], [170]. These APIs facilitate the seamless integration of many ML functionalities into applications. After receiving one query for one specific downstream task through API, the cloud-based server processes the query with one featured LLM model. Under this scenario, the proposed cloud solution for handling multiple PEFT queries involves storing only a single copy of the LLM and multiple PEFT modules. This single copy maintains multiple branches of PEFT modules, each associated with different PEFT queries. The case study of a state-of-the-art system can be found in

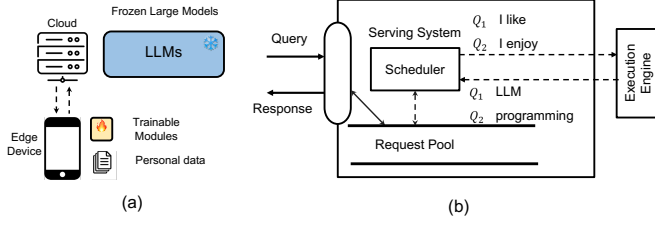


Fig. 10: (a) Distributed-based system computation pattern; (b) centralized PEFT Query inference

Section VI-C. Figure 10 (b) illustrates the computation pattern for multi-query PEFT inference, wherein packed PEFT queries are scheduled and executed according to their deadlines and current system conditions.

2) *Serving Metrics*: To evaluate the system performance of centralized PEFT query serving, we propose a set of evaluation metrics.

- **System throughput**: Considering PEFT queries as inter and intra tasks, we use tokens per second to measure the system throughput.
- **Memory footprint**: Run-time memory consumption during query serving, the memory utilization comes from both model parameters and KV-cache as mentioned in Section IV-A.
- **Accuracy performance**: Real-world queries normally have different context lengths, and performance with variation length serves as a performance benchmark.
- **Quality of services**: Queries are associated with latency requirements and deadline missing rates are considered as another benchmark.

3) *Distributed System for PEFT*: Nevertheless, in the contemporary LLM model, personalized tasks are not fully supported with pre-trained models, consequently, extra fine-tuning is required to be executed with the methodologies mentioned in the previous sections. However, a big concern is raised when we consider giving the datasets to cloud providers since these datasets are personalized.

For this concern, we assume our computing follows the model-centralized and PEFT-distributed paradigm. The backbone LLM is stored in the cloud devices while the personal PEFT weight as well as datasets are stored in users' own devices. This is depicted in Figure 10(a).

4) *Distributed Metrics*: To assess the efficacy of the proposed method, we establish a set of evaluative metrics. For this analysis, and without loss of generality, we adopt language models as the basis for our metric definitions.

- **Accuracy performance**: Performance of the fine-tuned model over the downstream tasks.
- **Compute cost**: The compute cost during forward and backward propagation operations on edge devices.
- **Communication cost**: Refers to the volume of data involved during the transfer of intermediate data between the edge device and the cloud.

5) *Multi-PEFT Training*: Different from multiple-PEFT serving, tuning with multiple customized PEFTs always involves different backbone LLMs. When contemplating LLM

Algorithm 1 KR Algorithm (simplified version)

```

1: Inputs: LLM module pool  $F$ ; Total number of layer  $L$ ;
   Selection criteria  $\epsilon$ . Total finetuning epoch  $E$ ;
2:  $\triangleright$  Pre-tuning phase
3: Tune all PEFT modules within  $F$  for several iterations to
   collect statistics.
4: Record the changes on  $l_2$  norms for each module, define
   the selection criteria.
5:  $\triangleright$  Main tuning phase
6: for  $0 \leq e \leq E - 1$  do
7:   for  $l \in L$  do
8:     if  $l_2$  norm change on  $l$ -th PEFT module less than
        $\epsilon$  then
9:       Frozen the PEFT module at layer  $l$ .
10:    else
11:      Activate  $l$ -th PEFT module.
12:      Finetune all active PEFT modules.
13:      Recalculate  $\epsilon$ .

```

usage across various downstream tasks, pre-trained models typically exhibit subpar performance. A prevalent approach to adapt LLM to diverse tasks involves crafting fine-tuned PEFTs. However, simultaneously tuning multiple PEFTs can pose considerable challenges. Challenges like how to manage memory gradient and model weights storage, and how to design an efficient kernel for batching PEFT training remain unsolved. PEFTs will be categorized based on their PEFT algorithms and backbone LLM models. The design challenge involves how to consolidate multiple PEFTs with the same LLM backbone and multiple different LLM backbones simultaneously.

B. Case study: DLoRA

Due to the scale of LLM, PEFT operations are usually executed in the public environment (e.g., cloud server). This necessitates the sharing of sensitive user data across public environments, thereby raising potential privacy concerns. To tackle these challenges, a solution called *DLoRA* is proposed. DLoRA enables scalable PEFT operations to be performed collaboratively between the cloud and user devices. The evaluation results demonstrate that DLoRA can significantly reduce the computation and communication workload over the user devices while achieving superior accuracy and privacy protection.

DLoRA relies on an algorithm 1 termed *Kill and Revive* (KR) to reduce user-end computation and cloud and user communication. DLoRA recorded different PEFT modules' weight changes in different decoders during the tuning session. Figure 11 shows the sampled PEFT module weights over 4 epochs. We observe that some PEFT modules change greatly while others remain stable. DLoRA proposes that edge-side PEFT modules can be killed when their weight changes are not noticeable and be revived when their weight changes are significant.

The cloud and user ends can conduct computation simultaneously. After receiving the activation for a specific PEFT module from the cloud, the user device will start computing

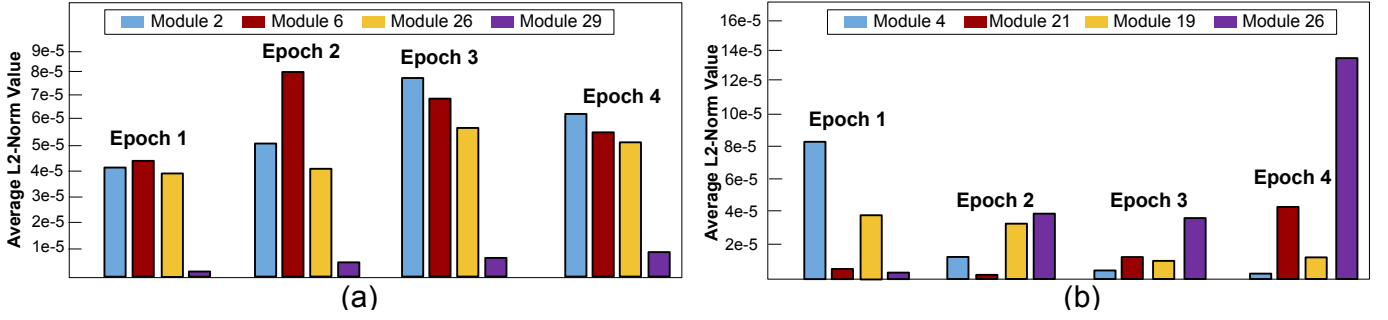


Fig. 11: l_2 -norm variation of selected PEFT modules across training iterations over multiple downstream tasks including (a) Arc-Challenge, (b) Social-QA.

output activation for the PEFT module. In the meantime, the cloud ends can continue computing other blocks. The synchronous barrier between users and the cloud is the activation functions (e.g. ReLU) in each decoder. This is because PEFT module computation normally performs residual block before each activation layer.

C. Case Study: PetS

The PEFT algorithm is notable for its ability to distinguish between modifiable and immutable weights within a model. This characteristic inspires developers to amalgamate diverse LLMs with distinct PEFT techniques into collective units. PetS, as introduced in [235], advocates for a comprehensive approach to managing multiple PEFT tasks by suggesting a unified serving framework. The framework's core advancement lies in the translation of varying PEFT tasks into integrated computation kernels to enhance efficiency. Moreover, PetS pioneers an orchestrated batching approach and a scheduling methodology, aiming to augment system throughput and leverage task parallelism respectively.

As depicted in Figure 12, the PetS framework begins with users registering PEFT tasks through a standardized Application Programming Interface (API). Upon registration, developers are expected to provide the Pre-Trained Model Tag (e.g., LLaMA), PEFT parameters in a compressed format, and the specific PEFT algorithms (e.g., LoRA, Adapter, Bitfit, etc.). These tasks are then endowed with unique identifiers, and the inference engine takes charge of query processing. PetS bifurcates the primary computational workload (e.g., linear layer computations) into three distinct computational operations: (1) Dense Matrix-Vector Multiplication (MVM) leveraging universally accessible, pre-trained weights. (2) Bias vector addition (Vadd), using either common or task-exclusive biases. (3) A combination of Sparse/dense MVM operations employing task-specific PET parameters. A unified pre-trained weight matrix W is employed across PetS, facilitating the batching of initial operations, $X_t \times W$. However, subsequent task-specific computations involving PET parameters, despite being relatively minimal in complexity, are processed individually.

Considering the Adapter and Bitfit tasks as an illustration, both aim at the MLP component of LLMs. The Adapter task integrates additional weight segments, whereas Bitfit

adjusts bias elements. The Adapter operation is modeled as $Y = X_{in1} \times (W + W_{ad}) + b_0$, where X_{in1} represents the input for the Adapter task, W and W_{ad} are the original and adapter-specific PEFT weights respectively, and b_0 is the initial bias. The Bitfit operation, on the other hand, is defined as $Y = X_{in2} \times W + b_1$, with b_1 symbolizing the Bitfit-adjustable bias. These operations are further synthesized as $\{Y_1, Y_2\} = \{X_{in1}, X_{in2}\} \times W + \{X_{in1} \times W_{ad}, 0\} + \{b_0, b_1\}$, delineating that the $\{X_{in1}, X_{in2}\} \times W$ part is amenable to batching through MVM, while the $\{b_0, b_1\}$ segment pertains to the Vadd operation.

For tasks like Diff-Pruning III-B, is a little bit different than Bitfit and Adapter. For Diff-Pruning, the computation concerning the shared weight and 'difference' are conducted separately. Then the results are added up, namely

$$X_t \times (W + \delta_t) = X_t \times W + X_t \times \delta_t$$

, here, the W denotes the backbone model weights while δ_t denotes the pruned weights which can be represented as Sparse MVM.

The other challenge PetS proposed is how to schedule different PEFT requests to achieve high performance. PetS scheduler achieves high parallelism through a two-level scheduling policy: Coordinated Batching (CB) and Macro-batch Streaming (MS) as Figure 13 depicts. Through CB, the input queries will first be clustered based on their input length and then grouped based on their shared operator. This is to make sure the same sequence length of queries will be executed without wasting padding. MS strategy will take the grouped queries after coordinated batching and the theoretical latency for different operators as well as the system modeling parameters to generate the best execution order.

D. Parallel PEFT Training Frameworks

a) *Design Challenges:* Unlike the PetS system, which aims to accommodate flexible multi-PEFT algorithms, S-LoRA [158] and Punica [19] focus solely on facilitating multiple-LoRA blocks for various tasks. Designing multiple PEFT training systems presents key challenges in two main aspects:

- Efficient concurrent execution of multiple PEFT models with the same LLM backbone.
- Designing an efficient system for multi-tenant serving with different LLM backbones.